

Sequential Utility Mining with the Maximum Measure

Guo-Cheng Lan^a, Tzung-Pei Hong^{b,c} and Vincent S. Tseng^{a,d}

^aDepartment of Computer Science and Information Engineering
National Cheng Kung University, 701, Taiwan

^bDepartment of Computer Science and Information Engineering
National University of Kaohsiung, 811, Taiwan

^cDepartment of Computer Science and Engineering
National Sun Yat-Sen University, 804, Taiwan

^dInstitute of Medical Informatics

National Cheng Kung University, 701, Taiwan

rrfoheiay@gmail.com; tphong@nuk.edu.tw; tsengsm@mail.ncku.edu.tw

Abstract

In this paper, a new measure, namely maximum utility measure, is designed to evaluate the utility of a pattern in a sequence. Based on the measure, a projection-based algorithm for finding high-utility sequential patterns from a sequence database is developed. Experimental results show the performance of the proposed algorithm as well.

1 Introduction

Sequential pattern mining [2] on data mining has been widely applied to trend analysis from a set of long-term event sequence data. A sequential pattern, which its frequency is larger than or equal to a predefined support threshold in a sequence database, can provide the order of events (items) in the pattern. However, only the occurrence of items in traditional sequential pattern mining is considered in sequences, and itemsets found in this way do not reflect any other factors, such as cost, price or profit. For example, assume a buying behavior pattern like that customers also bought DVD players within one month after they bought LCD TVs. The buying behavior pattern may not be a frequent pattern in the sales database of a supermarket, but the pattern may contribute a large portion to the overall profit due to LCD TVs with high-profit. As this example illustrates, some products with high profit but low frequency may not be discovered in a sequence database by using traditional sequential pattern mining approaches.

To deal with the problem of sequential patterns with low-frequency but high-profit, Ahmed *et al.* proposed a new issue, namely utility sequential pattern mining. In Ahmed *et al.*'s study [3], they not only considered quantities of items in a sequence but also individual profits of items in the sequence database. Then, a utility sequential

pattern with actual utility, which satisfies a predefined sequence utility threshold, could be found from a sequence database. In Ahmed *et al.*'s study [3], if a pattern in a sequence may appear multiple times, then the utility of the pattern in the sequence is the summation of utility values of all distinct occurrences of the pattern in the sequence. In addition, if multiple distinct occurrences of the pattern in a sequence cannot be formed, the utility of the pattern in the sequence is given by taking the maximum utility of the pattern in the sequence. Based on the above definitions, it could be clearly known that utility of a pattern in a sequence is overly overestimated.

In traditional sequential pattern mining, although a pattern in a sequence may appear multiple times, the count of the pattern in the sequence is only given to one. Based on the main principle of sequential pattern mining, the maximum utility measure is suitable for giving the utility of a pattern in a sequence since the maximum utility of a pattern in a sequence can be represented that a user is willing to contribute the largest utility at a time for the pattern in the user's historic buying records. The measure is not overly overestimated utility of a pattern when compared with Ahmed *et al.*'s definitions [3].

However, downward-closure property on traditional sequential pattern mining cannot be kept in the proposed utility sequential pattern mining with the maximum utility measure. To deal with this, we thus propose a sequence utility upper-bound (abbreviated as *SUUB*) model to find all high-utility sequential patterns from a sequence database. In addition, we propose an efficient projection-based mining approach (named *PHS*) for achieving the goal of finding high-utility sequential patterns.

The remaining parts of this paper are organized as follows. Some related works are reviewed in

Section 2. The problem to be solved and its related definitions are described in Section 3. The proposed algorithm for finding high-utility sequential patterns from a sequence database is stated in Section 4. An experimental evaluation is shown in Section 5, and conclusions are finally given in Section 6.

2 Review of related works

Association-rule mining on data mining only considers the items whether they appear in transactions or not [1]. However, in real-world applications, the transaction time of each transaction is usually recorded in databases. These transactions can then be listed as a time-series data (called sequence data) in their occurring time order, and then buying behavior patterns can be found from the sequence data. Sequential pattern mining was thus proposed to discover such patterns from a set of sequence data [2]. However, since the three algorithms in the study [2] adopted the level-wise technique, the algorithms had to execute multiple data scans to complete the sequential pattern mining task. Thus, many other algorithms were subsequently proposed to improve execution efficiency in terms of finding sequential patterns, such as *PrefixSpan* [8].

On the other hand, some other researches then considered weight values or quantities of items in a sequence database to recognize importance of the items. Yun developed several techniques to find weighted sequential patterns from sequence databases [10]. In Yun's study [10], different items were then given different weight values. In addition, Kim *et al.* proposed a SQUIRE algorithm for finding sequential patterns with quantity constraint [7], which the supports of the patterns have to satisfy a predefined minimum support threshold. However, the SQUIRE algorithm [7] only considered quantities of items in sequences, but did not consider other important factors of items, such as profit, or cost.

In traditional utility mining [9], a utility itemset not only considered quantities of items in a transaction but also individual profits of items in the transaction database. Due to its practical applications, utility mining has been applied to various problems, such as on-shelf utility mining [5]. However, utility mining techniques were not applied to handle a sequence data. To deal with this, Ahmed *et al.* proposed a new issue, namely utility sequential pattern mining. In Ahmed *et al.*'s study [3], they not only considered quantities of items in a sequence but also individual profits of items in the sequence database. A utility sequential pattern with actual utility, which satisfied a predefined utility threshold, could be found and

output as a high-utility sequential pattern desired by users.

According to Ahmed *et al.*'s definitions [3], if a pattern in a sequence has multiple distinct occurrences, then the utility of the pattern in the sequence is the summation of utility values of all distinct occurrences of the pattern in the sequence. Besides, if multiple distinct occurrences of the pattern in a sequence cannot be formed, then the utility of the pattern in the sequence is given by taking the maximum utility of the pattern in the sequence. Based on the definitions in Ahmed *et al.*'s study [3], it could be known that the utility of a pattern in a sequence is overly overestimated.

However, by the principle of traditional sequential pattern, a pattern in a sequence may appear multiple times, the count of the pattern in the sequence is only given to one. The maximum utility of a pattern in a sequence can be represented that a user is willing to contribute the largest utility at a time for the pattern in the user's historic buying records. In this study, the maximum utility measure is thus developed to give the utility values of patterns in sequences.

3 Problem statement and definitions

To clearly describe the problem to be solved, assume a sequence database is given in Table 1, in which each sequence consists of two features, sequence identification (*SID*) and items purchased. There are seven items in the sequences, respectively denoted as *a* to *g*. The value attached to each item in the corresponding slot is the quantity sold in a sequence. Also, assume the profit value of each item is shown in Table 2.

TABLE 1
The set of six sequence data in this example

<i>SID</i>	<i>Sequences</i>
s_1	$a(3) \{a(2) b(2) d(6)\} f(1) a(5) d(1)$
s_2	$e(3) \{a(2) b(1)\} d(5) c(4)$
s_3	$\{c(1) f(2)\} b(1) \{d(3) e(4)\}$
s_4	$a(2) \{b(2) d(7)\} \{a(6) d(4)\} e(5)$
s_5	$\{d(3) f(3)\} c(5) g(2)$
s_6	$d(8) e(1) \{a(7) b(2)\} b(3) d(6) e(3)$

TABLE 2:
The predefined profit values of the items

<i>Item</i>	<i>Profit</i>
<i>a</i>	3
<i>b</i>	10
<i>c</i>	1
<i>d</i>	6
<i>e</i>	5
<i>f</i>	2
<i>g</i>	8

For the utility sequential pattern mining problem to be solved, a set of terms related to utility sequential mining, which is extended from utility mining [6], is defined as follows.

Definition 1. $I = \{i_1, i_2, \dots, i_m\}$ is a set of items, which may appear in transactions.

Definition 2. An itemset X is a subset of items sorted alphabetically, $X \subseteq I$. If $|X| = r$, the itemset X is called an r -itemset. For example, the itemset $\{ab\}$ contains 2 items and is called a 2-itemset.

Definition 3. $s = \langle X_1, X_2, \dots, X_m \rangle$ is a list of itemsets sorted in ascending order according to their occurrence time. Each itemset appears in a sequence is called an element of the sequence. For brevity, the brackets are omitted if an itemset only includes an item. For example, in Table 1, the second element in the fifth sequence s_5 only includes the item c .

Definition 4. A sequence database SDB is composed of a set of sequences. That is, $SDB = \{s_1, s_2, \dots, s_y, \dots, s_z\}$, where s_y is the y -th transaction in SDB .

Definition 5. The local sequence utility $o(i_{yj}, s_y)$ of an item i_{yj} in a sequence s_y is the quantity of i_{yj} in s_y . For example, in Table 1, $lsu(b, s_1) = 2$.

Definition 6. The external utility $s(i_j)$ of an item i_j is the corresponding utility value of the item in the utility table. For example, $s(a) = 3$ in Table 2.

Definition 7. The utility $u(i_{yj}, s_y)$ of an item i_j in a sequence s_y is the external utility $s(i_j)$ of the item i_j in the utility table multiplied by the maximum local sequence utility $lsu(i_{yj}, s_y)$ of the item i_j in the sequence s_y , and can be defined as $u(i_{yj}, s_y) = s(i_j) * lsu(i_{yj}, s_y)$. For example, $u(a, s_1) = 3 * 5 = 15$ in Table 1.

Definition 8. The utility $u(X_y, s_y)$ of a sequence X in a sequence s_y is the maximum utility X in s_y . For example, $u(\langle aba \rangle, s_1) = u(a, s_1) + u(b, s_1) + u(a, s_1) = 5 * 3 + 2 * 10 + 3 * 3 = 44$ in Table 1.

Definition 9. The actual sequence utility $asu(X)$ of a sequence X in a database SDB is the sum of the utility values of X in all the sequences of SDB . For example in Table 1, $asu(aba) = u(aba, s_1) + u(aba, s_4) = 44 + 44 = 88$.

Definition 10. Let λ be a pre-defined minimum sequence utility threshold. A sequence X is called a high-utility sequential pattern (abbreviated as *HUS*) if $asu(X) \geq \lambda$. For example in Table 1, if $\lambda = 85$, then the sequence $\langle aba \rangle$ is a high-utility sequential pattern since $asu(\langle aba \rangle) = 88 \geq \lambda$.

However, utility sequential mining does not have the downward-closure property. Thus, a new sequence utility upper-bound (abbreviated as *SUUB*) model is proposed to find all high utility sequential patterns with maximum utility property from a sequence database. Here some definitions related to the *SUUB* model are described below.

Definition 11. The sequence utility $su(s_y)$ of a

sequence s_y is the sum of the utility values of the items contained in s_y . That is,

$$su(s_y) = \sum_{i_j \in s_y} u(i_j, s_y).$$

For example in Table 1, $su(s_5) = u(d, s_5) + u(f, s_5) + u(c, s_5) + u(g, s_5) = 3 * 6 + 3 * 2 + 5 * 1 + 2 * 8 = 45$.

Definition 12. The sequence utility upper-bound $suub_X$ of a sequence X is the summation of the sequence utilities of all the sequences containing X in SDB . That is,

$$suub(X) = \sum_{s_y \in SDB \wedge X \subseteq s_y} su(s_y).$$

For example in Table 1, $suub(\langle a \rangle) = su(s_1) + su(s_2) + su(s_4) + su(s_6) = 94 + 65 + 135 + 175 = 469$.

Definition 13. Let λ be a pre-defined minimum sequence utility threshold. A sequence X is called a high sequence utility upper-bound pattern (abbreviated as *HSUUB*) if $suub(X) \geq \lambda$. For example in Table 1, if $\lambda = 85$, then the sequence $\langle a \rangle$ is a high sequence utility upper-bound 1-pattern.

Based on the definitions above, a utility sequential pattern considers the individual profits and quantities of items in sequence data by the maximum utility measure. The problem to be solved in the paper is to find all high-utility sequential patterns, which their actual sequence utility are larger than or equal to a predefined minimum sequence utility threshold, from a given sequence database. The details of the proposed projection-based algorithm for finding high-utility sequential patterns (abbreviated as *PHS*) are described in the next section.

4 The proposed algorithm

In this section, we propose an efficient mining algorithm to find high-utility sequential patterns with the maximum utility measure by using the maximum utility measure. The details of the proposed projection-based algorithm for finding high-utility sequential patterns (*PHS*) are then stated as follows.

INPUT: A set of items, each with a profit value; a sequence database SDB , in which each sequence includes a subset of items with quantities; a minimum sequence utility threshold λ .

OUTPUT: A final set of high-utility sequential patterns (*HUSP*).

STEP 1: For each y -th sequence s_y in SDB , do the following substeps.

- (a) Calculate the utility value $u(i_{yj}, s_y)$ of each j -th item i_{yj} in s_y .
- (b) Calculate the sequence utility $su(s_y)$ of the

- sequence s_y .
- STEP 2: Calculate the total sequence utility $tsu(SDB)$ for all sequences in SDB .
- STEP 3: Find the actual sequence utility $asu(i_j)$ and the sequence utility upper-bound $suub(i_j)$ of each item i_j in SDB ,
- STEP 4: If the sequence utility upper-bound $suub(i_j)$ of an item i_j is larger than or equal to the minimum sequence utility threshold λ , put it in the set of high sequence utility upper-bound 1-patterns, $HSUUB_1$; if its actual sequence utility $asu(i_j)$ is larger than or equal to λ , put it in the set of high-utility sequential 1-patterns, HSU_1 .
- STEP 5: Acquire the items appearing in the set of $HSUUB_1$ and denote them as S_1 , and sort all the items in the set of S_1 in alphabetical order of the items.
- STEP 6: Set $r = 1$, where r represents the number of items in the current set of candidate patterns (C_r) to be processed.
- STEP 7: For each s_y in SDB , do the following substeps.
- (c) Remove the items not appearing in the set of S_1 in s_y .
 - (d) If the number of items kept in s_y is smaller than the value $(= r + 1)$, remove the sequence s_y ; otherwise, keep it in SDB .
- STEP 8: Process each 1-pattern x in order from the last to the first in the sorted set of S_1 by the following substeps.
- (e) Find relevant sequences including x in SDB , and only x and the items located after x in that sequence are projected and put in the set of sequences d_x .
 - (f) Find all the high-utility sequential patterns with the item x as their prefix item with the *Finding-HUS*(x, d_x, r) procedure. Let the set of returned high-utility sequential patterns be HUS_x .
- STEP 9: Output the set of high-utility sequential patterns in all the HUS_x .

After STEP 9, all high-utility sequential patterns are found. The *Finding-HUS* (x, d_x, r) procedure used in the proposed algorithm is stated, as follows.

The *Finding-HUS* (x, d_x, r) procedure:

INPUT: A prefix r -pattern x and its projected sequences d_x .

OUTPUT: The high-utility sequential patterns (HSU).

- PSTEP 1: Initialize the temporary sequence table as an empty table, in which each tuple

consists of three fields: pattern, sequence utility upper-bound ($suub$), and actual sequence utility (asu).

- PSTEP 2: Do the following substeps for each sequence s_y in d_x .

- (a) Get each item i_{yj} located after the prefix r -itemset x in s_y .
- (b) Generate the $(r+1)$ -pattern x' composed of the prefix r -pattern x and i_{yj} , put it in the temporary sequence table, and add its sequence utility $su(s_y)$ and utility $u(x', s_y)$ in the corresponding fields in the temporary sequence table by using the maximum utility measure.

- PSTEP 3: If the sequence utility upper-bound $suub(x')$ of each $(r+1)$ -pattern x' in the temporary sequence table is larger than or equal to λ , put it in the set of $HSUUB_{r+1}$ with x as their prefix; if the actual sequence utility $asu(x')$ is larger than or equal to λ , put it in the set of HSU_{r+1} with x as their prefix.

- PSTEP 4: Acquire the items appearing in the set of $HSUUB_{r+1}$ with x as their prefix and denote them as S_{r+1} .

- PSTEP 5: Do the following substeps for each sequence s_y in d_x .

- (a) Remove the items not appearing in the set of S_{r+1} from s_y .
- (b) Check whether the number of items kept in s_y is smaller than the value $(= r+2)$. If it is, remove the sequence s_y ; otherwise, keep it in d_x .

- PSTEP 6: Do the following substeps for each pattern x' in the set of $HSUUB_{r+1}$ with the prefix x .

- (a) Find the sequences including x' in d_x , where x' and the items located after x' in each sequence are projected in the new sequence, and put it in $d_{x'}$.
- (b) If the number of items kept in each projected sequence of d_s is smaller than the value $(= r + 2)$, remove the sequence; otherwise, put it in $d_{x'}$.
- (c) Find all the high-utility sequential patterns with x' as their prefix pattern by the *Finding-HUS*($x', d_{x'}, r+1$) procedure. Let the set of high-utility sequential patterns returned be $HUS_{x'}$.

- PSTEP 7: Return the set of high-utility sequential patterns in all the $HUS_{x'}$.

5 Experimental Evaluation

All the algorithms were implemented in J2SDK 1.5.0 and executed on a PC with 3.0 GHz CPU and 1 GB memory. The public *IBM* data generator was used in our experiments to produce the data

sets [4]. Since our purpose was to find high-utility sequential patterns, we developed a simulation model, which was similar to that used in Liu *et al.* [6]. The model is extended to generate the quantity values of the items in the sequences. Each quantity ranged among 1 to 5 following the way described in [6]. In addition, for each dataset generated, a corresponding utility table was also produced in which a profit value in the range from 0.01 to 10.00 was randomly assigned to an item. Figure 1 showed the number of high-utility sequential patterns discovered by *PHS* and *UL* [3] on the S815N10KD200K dataset under various sequence utility thresholds, varying from 0.10% to 0.02%.

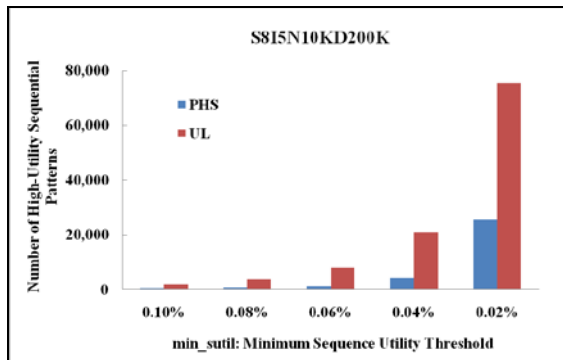


Figure 1: The comparison of the numbers of patterns discovered by the two algorithms along with various thresholds, *min_util*.

It could be seen in the figure that the number of high-utility sequential patterns discovered by the proposed algorithm *PHS* was obviously less than that discovered by the existing algorithm *UL*. The main reason for this was that the maximum utility measure used in *PHS* was overly overestimated the utility values of patterns in sequences when compared with the model used in *UL*.

6 Conclusions

In this paper, we have proposed a new utility measure for utility sequential pattern mining, and an efficient projection-based mining approach (*PHS*) is also proposed to find high-utility sequential patterns in sequence databases. The experimental results show that the number of high-utility sequential patterns required by the proposed *PHS* algorithm is less than that required by the existing *UL* algorithm [3].

References

[1] R. Agrawal and R. Srikant, "Fast algorithm for mining association rules," *The International Conference on Very Large Data Bases*, pp. 487-499, 1994.

[2] R. Agrawal and R. Srikant, "Mining sequential patterns," *The 11th International Conference Data Engineering*, pp. 3-14, 1995.

[3] C. F. Ahmed, S. K. Tanbeer, and B. S. Jeong, "A novel approach for mining high-utility sequential patterns in sequence databases," *ETRI Journal*, Vol. 32, No. 5, pp. 676-686, 2010.

[4] IBM Quest Data Mining Project, "Quest synthetic data generation code," Available at (<http://www.almaden.ibm.com/cs/quest/syndata.html>).

[5] G. C. Lan, T. P. Hong, and Vincent S. Tseng, "Discovery of high utility itemsets from on-shelf time periods of products," *Expert Systems with Application*, Vol. 38, No. 5, pp. 5851-5857, 2011.

[6] Y. Liu, W. Liao, and A. Choudhary, "A fast high utility itemsets mining algorithm," *The Utility-Based Data Mining Workshop*, pp. 90-99, 2005.

[7] C. Kim, J. H. Lim, R. T. Ng, and K. Shim, "SQUIRE: sequential pattern mining with quantities," *J. Syst. Software*, Vol. 80, No. 10, pp. 1726-1745, 2007.

[8] J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M. Hsu, "Mining sequential patterns by pattern-growth: The PrefixSpan approach," *IEEE Transactions on Knowledge and Data Engineering*, Vol. 16, No. 11, pp. 1424-1440, 2004.

[9] H. Yao, H.J. Hamilton, and C.J. Butz, "A foundational approach to mining itemset utilities from databases," *The 3rd SIAM International Conference on Data Mining*, pp. 482-486, 2004.

[10] U. Yun, John J. Leggett, and T. J. Ong, "Mining weighted sequential patterns based on length-decreasing support constraints," *The International Conference on Intelligence and Security Informatics*, pp. 650-651, 2006.